МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Институт информационных технологий
Кафедра квантовых информационных технологий, практической и прикладной информатики

ОТЧЕТ ПО БЛОКУ ПРАКТИЧЕСКИХ РАБОТ №2

По дисциплине

Системы хранения данных

Работа выполнена	« »_____2026г.	Дьяков А.Г.
Зачтено	« »_____2026г.	Павлова Е.С.

Москва, 2026г.

Цель работы: освоить методы построения сетей хранения данных с применением протоколов iSCSI, NFS, SMB и FTP; настроить и протестировать взаимодействие клиента и сервера хранения данных по указанным протоколам.

1 ХОД РАБОТЫ

Стенд построен в VirtualBox и состоит из двух виртуальных машин под управлением Ubuntu 26.04.1 LTS (ядро Linux 7.0.0-38-generic): сервера хранения данных без графической оболочки (имя хоста andrew-dyakov, адрес 192.168.100.10) и клиента с графической оболочкой (имя хоста andrew-dyakov-VirtualBox, адрес 192.168.100.20). Во всех тестовых файлах записаны фамилия, имя и номер группы: «Дьяков Андрей, ИВБО-20-24».

1.1 Часть I. Установка и настройка виртуального стенда

На обе виртуальные машины установлена ОС Ubuntu. К серверу подключён дополнительный виртуальный диск ёмкостью 5 ГБ (устройство /dev/sdb), на основе которого создан том хранения средствами LVM: физический том (pvcreate /dev/sdb), группа томов vg_dyakov (vgcreate) и логический том lv_dyakov.

Первая попытка создать том командой lvcreate -L 5G завершилась ошибкой: в группе томов доступно 1279 экстентов вместо требуемых 1280 (часть диска занимают служебные метаданные LVM). Поэтому том создан командой lvcreate -l 100%FREE; его размер по выводу lvs составляет <5.00g, то есть соответствует заданным 5 ГБ. Затем том отформатирован в ext4 (mkfs.ext4), результат подтверждён командами lvs и lsblk (рисунок 1). При подключении по iSCSI клиент создаёт собственную файловую систему на этом устройстве (рисунок 11).

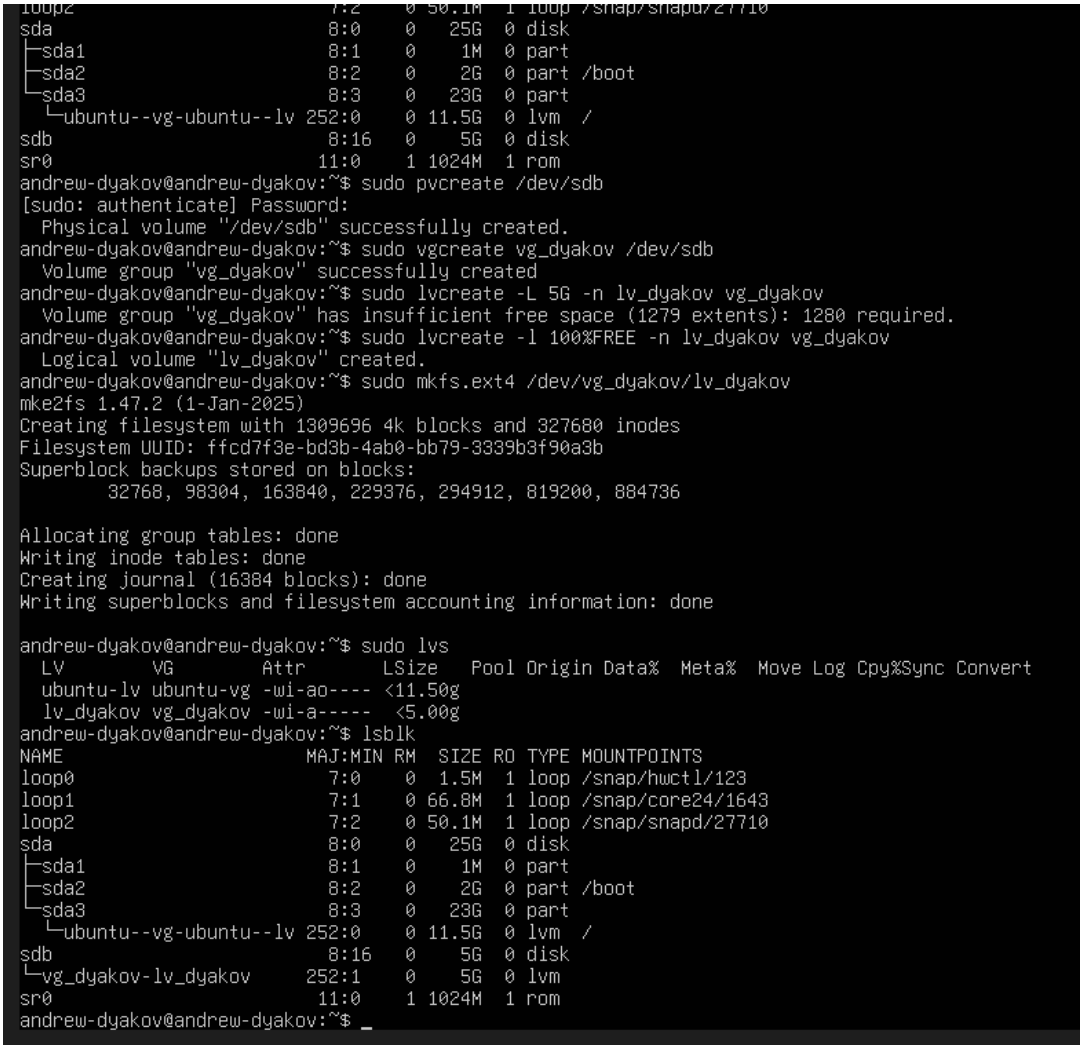


Рисунок 1 – Создание тома 5 ГБ на сервере средствами LVM и форматирование в ext4

Сетевая связность стенда проверена подключением клиента к серверу по SSH (ssh andrew-dyakov@192.168.100.10): принят отпечаток ключа ED25519, вход выполнен успешно (рисунок 2).

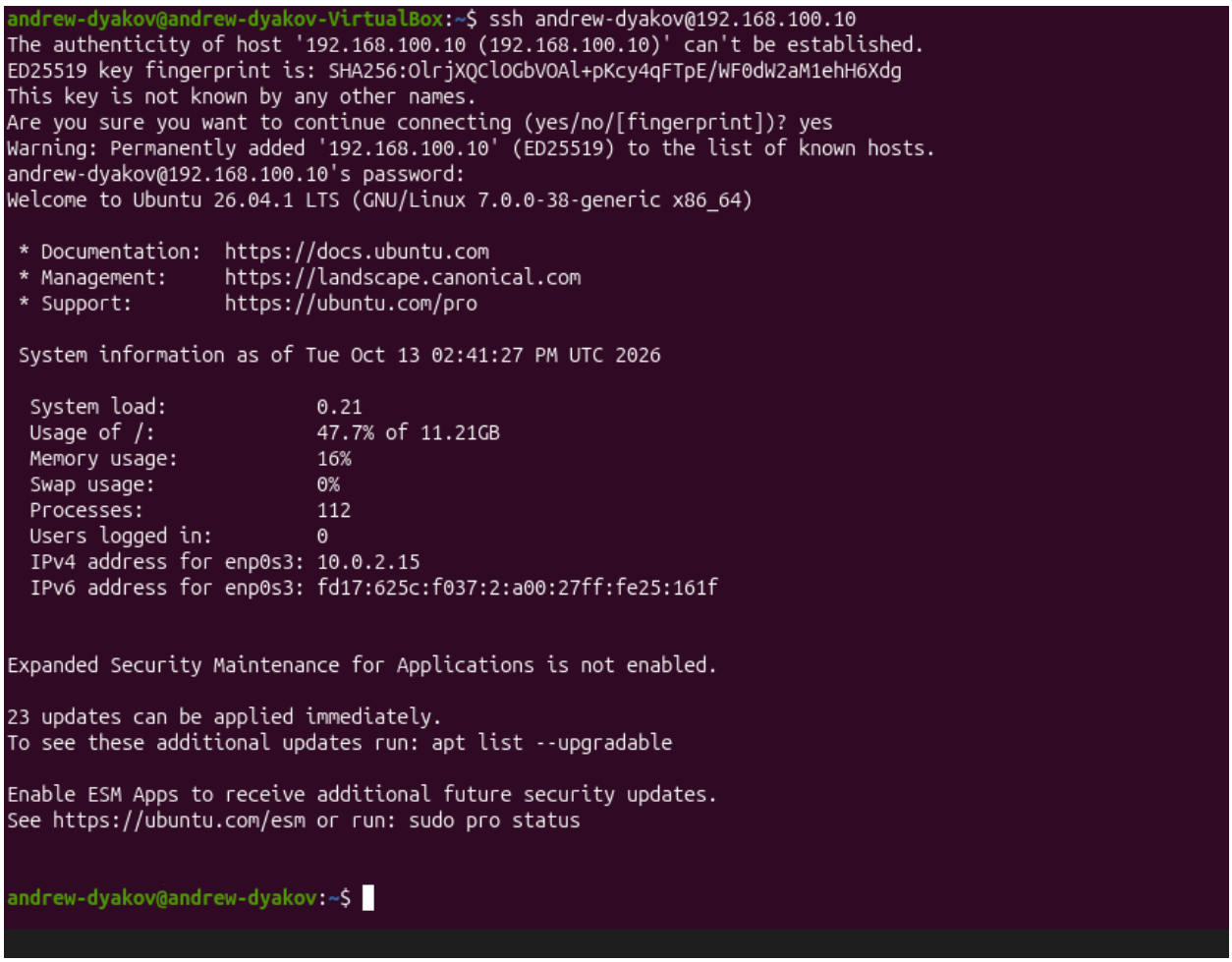


Рисунок 2 – Проверка подключения клиента к серверу по SSH

1.2 Часть II. Протокол iSCSI

В качестве iSCSI target на сервере использован пакет targetcli-fb (подсистема LIO ядра Linux) вместо устаревшего iscsitarget, а в качестве initiator на клиенте – пакет open-iscsi. Целевое имя (IQN): iqn.2026-10.local.storage:dyakov-target1.

Настройка сервера выполнена в следующем порядке. Установлен пакет targetcli-fb (рисунок 3). Логический том зарегистрирован как блочное хранилище (backstore) dyakov_lv (рисунок 4). Создан iSCSI target с группой портала TPG 1; по умолчанию портал слушает все адреса (::0) на порту 3260 (рисунок 5).

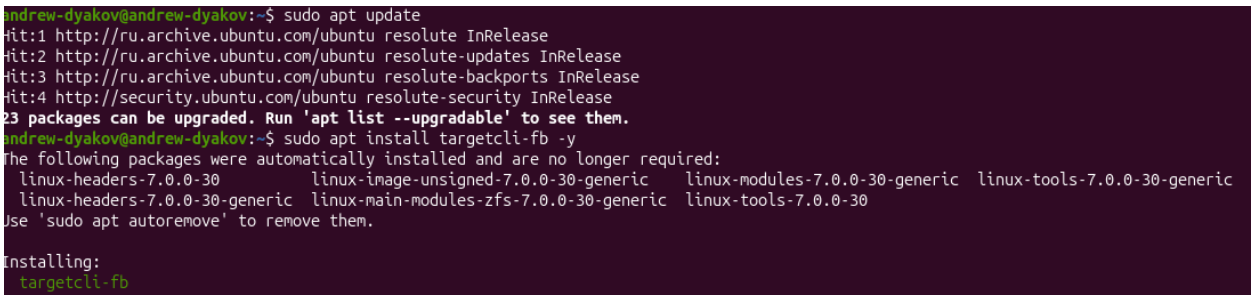


Рисунок 3 – Установка пакета targetcli-fb на сервере

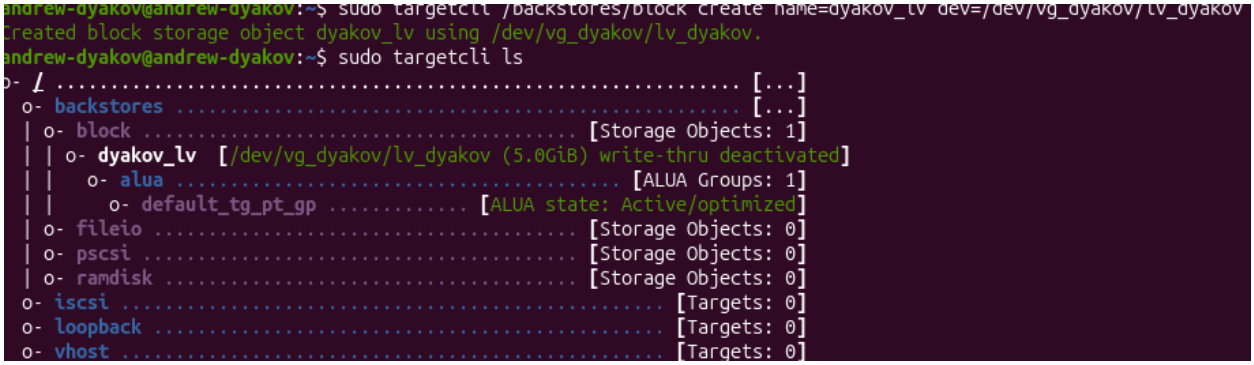


Рисунок 4 – Создание блочного хранилища dyakov_lv на основе логического тома

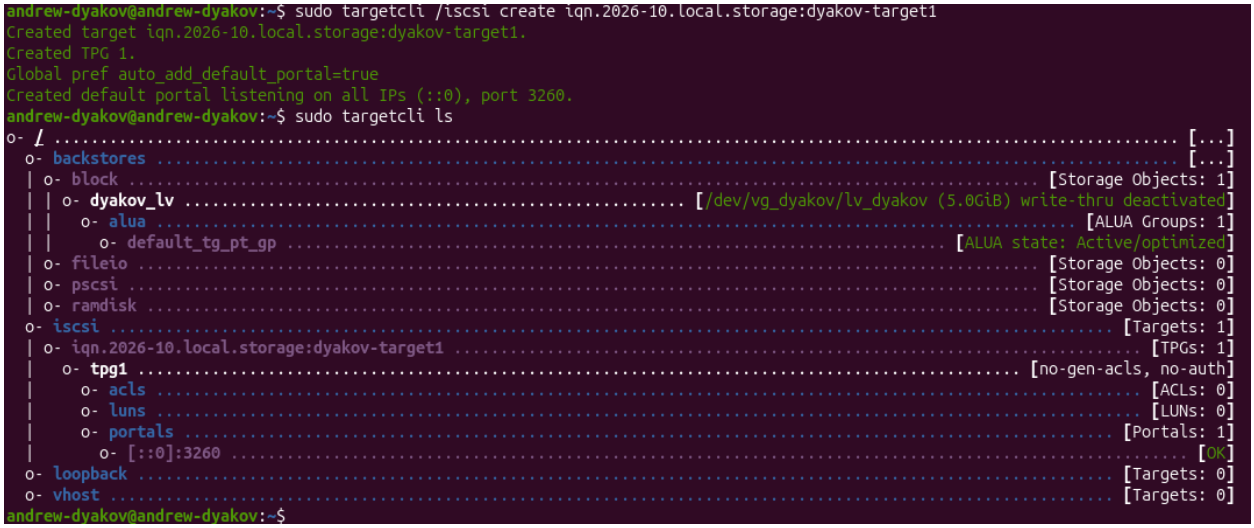


Рисунок 5 – Создание iSCSI target с именем IQN

Команда ss -tlnp | grep 3260 подтвердила, что порт 3260 прослушивается. Попытка добавить портал 192.168.100.10:3260 завершилась сообщением об ошибке, так как на этом порту уже существовал портал по умолчанию. К target подключён LUN 0 (luns create /backstores/block/dyakov_lv); параметры хранилища (устройство /dev/vg_dyakov/lv_dyakov, режим чтения-записи, WWN) показаны командой info (рисунок 6).

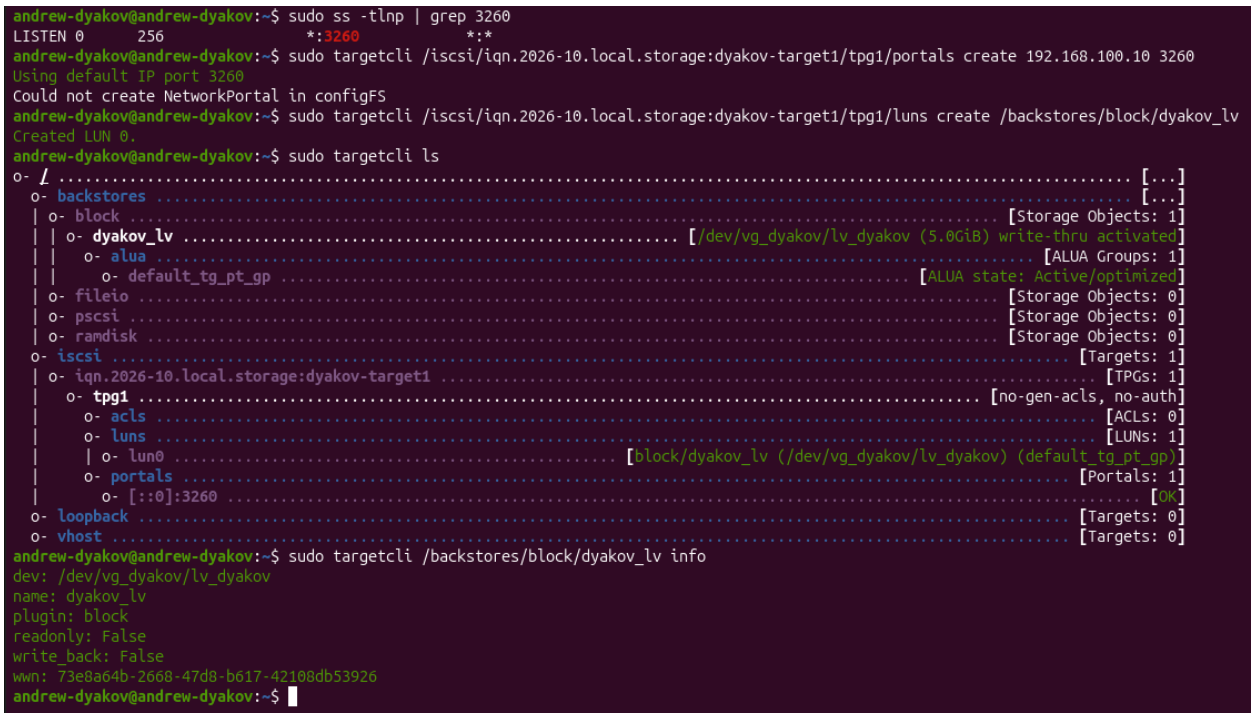


Рисунок 6 – Создание LUN 0 и проверка прослушивания порта 3260

Для авторизации на уровне iSCSI настроен CHAP: создан ACL для IQN инициатора iqn.2004-10.com.ubuntu:01:ad23a21b1ce2 с отображённым LUN 0, заданы имя пользователя и пароль CHAP, на группе портала включён атрибут authentication=1, конфигурация сохранена командой saveconfig. В итоговой конфигурации target работает в режиме «1-way auth», портал привязан к адресу 192.168.100.10:3260 (рисунок 7).

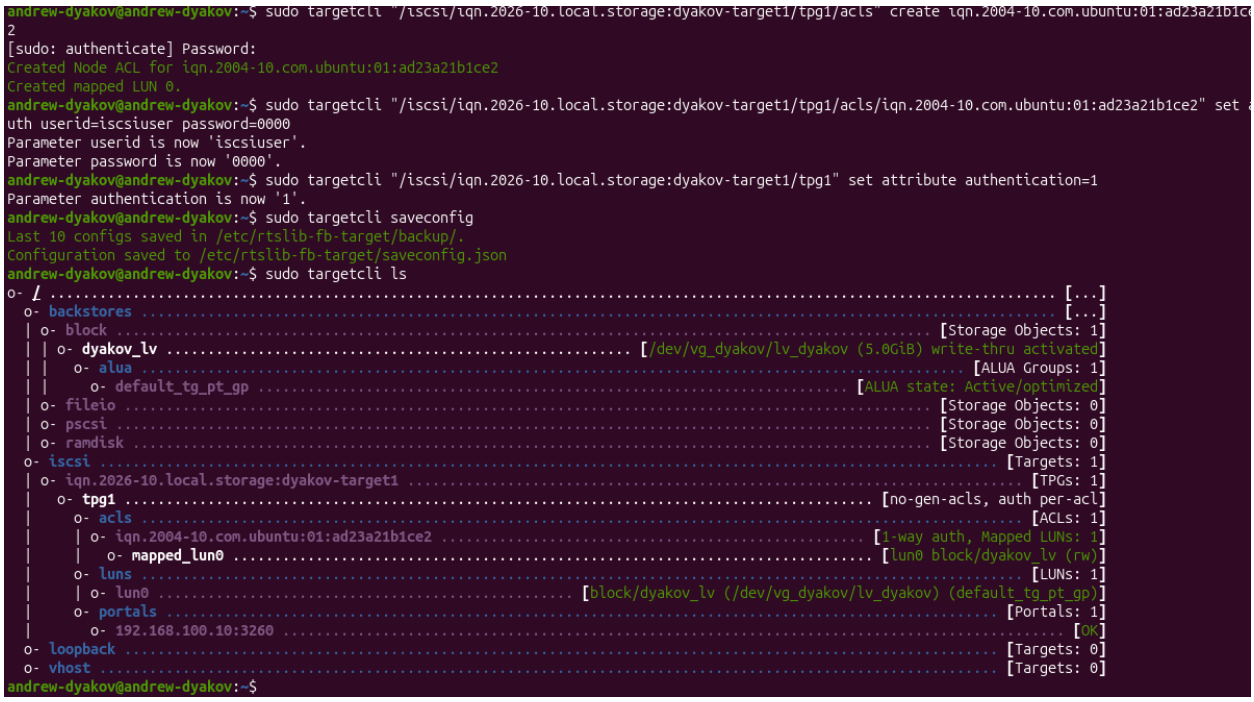


Рисунок 7 – Настройка ACL инициатора и CHAP-аутентификации на сервере

Настройка клиента. Пакет open-iscsi (версия 2.1.11-3ubuntu3.1) уже установлен в системе (рисунок 8). Командой iscsiadm -m discovery -t sendtargets -p 192.168.100.10:3260 обнаружен target. Для узла заданы метод аутентификации CHAP (node.session.auth.authmethod), имя пользователя и пароль, после чего выполнен вход (--login); вывод iscsiadm -m session показывает активную TCP-сессию с target (рисунок 9).

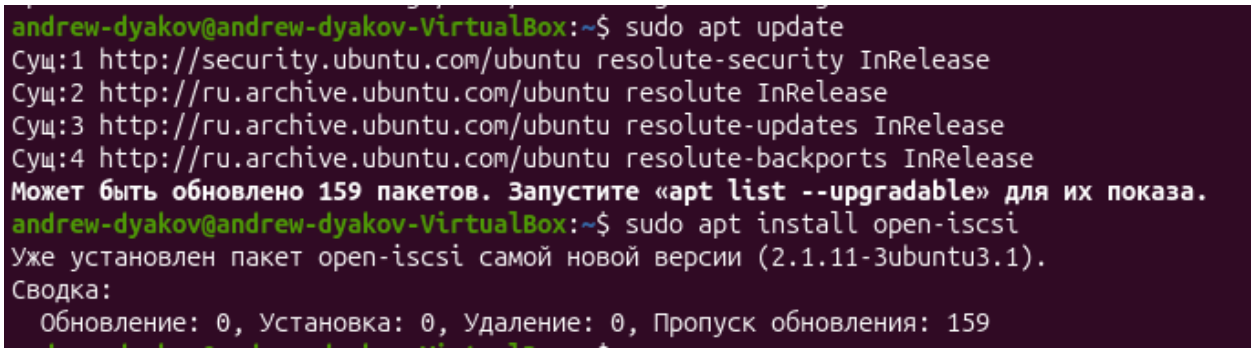


Рисунок 8 – Проверка наличия пакета open-iscsi на клиенте

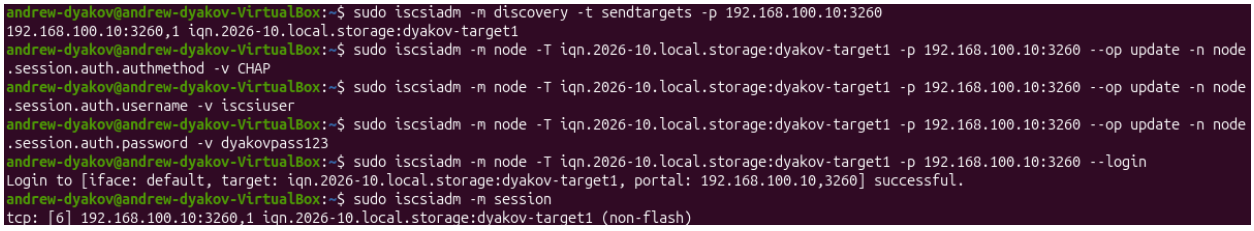


Рисунок 9 – Обнаружение target, настройка CHAP и подключение клиента к серверу

Команда lsblk показала новый диск sde ёмкостью 5 ГБ с моделью dyakov_lv – это LUN, предоставленный сервером (рисунок 10). Диск использован целиком, без таблицы разделов, поэтому вместо /dev/sdaX из методических указаний указано /dev/sde.



Рисунок 10 – Определение диска sde (модель dyakov_lv), предоставленного сервером

Том отформатирован в ext4 (mkfs.ext4 /dev/sde) и смонтирован в каталог /mnt/iscsi; df -hT показывает файловую систему ext4 размером 4,9 ГБ. В смонтированном томе создан тестовый файл testfile.txt с фамилией, именем и номером группы, его содержимое проверено командой cat (рисунок 11).

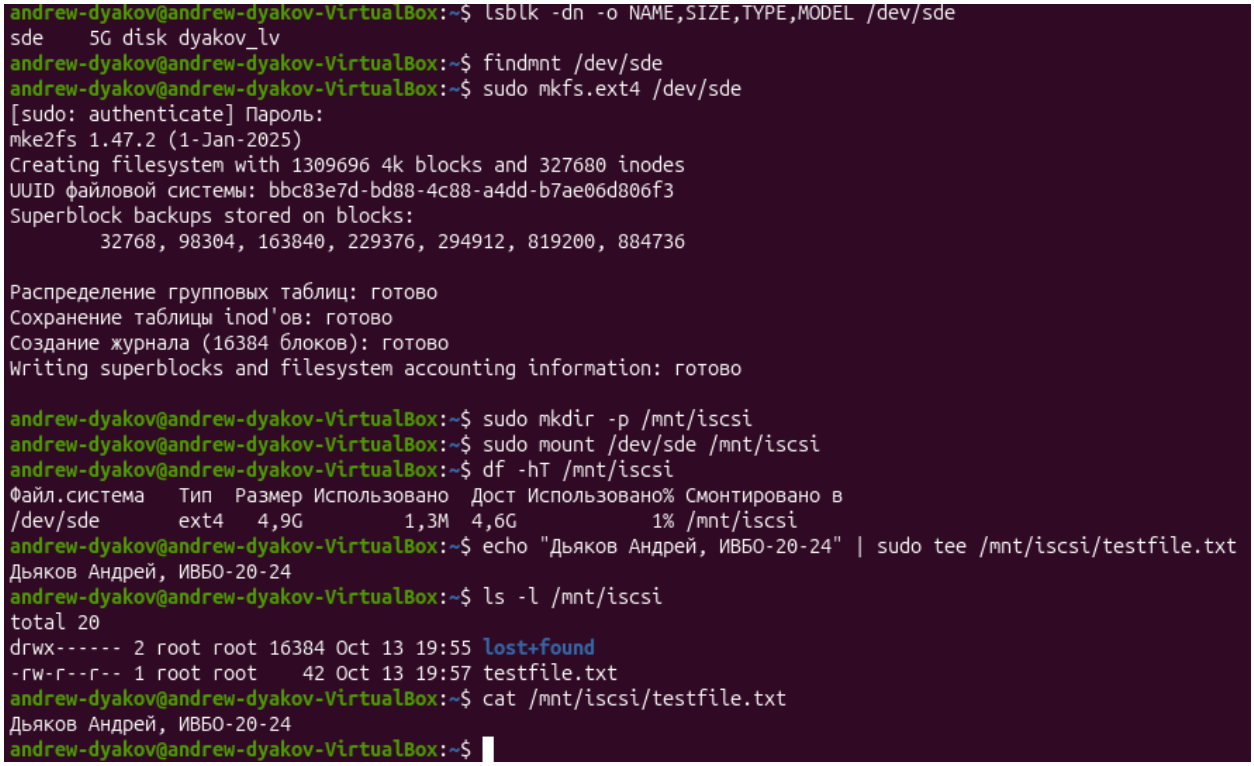


Рисунок 11 – Форматирование iSCSI-тома в ext4, монтирование и создание тестового файла

1.3 Часть III. Протокол NFS

На сервере установлен пакет nfs-kernel-server, создан общий каталог /shared_folder с правами 777 и добавлен экспорт в /etc/exports: /shared_folder

192.168.100.0/24(rw,sync,no_subtree_check,root_squash). В отличие от варианта * из методических указаний, доступ ограничен подсетью стенда, а запросы пользователя root клиента отображаются на непривилегированного пользователя (root_squash). Экспорт применён командой exportfs -ra, служба nfs-server перезапущена; статус active (exited) и enabled соответствует штатной работе службы (рисунок 12).

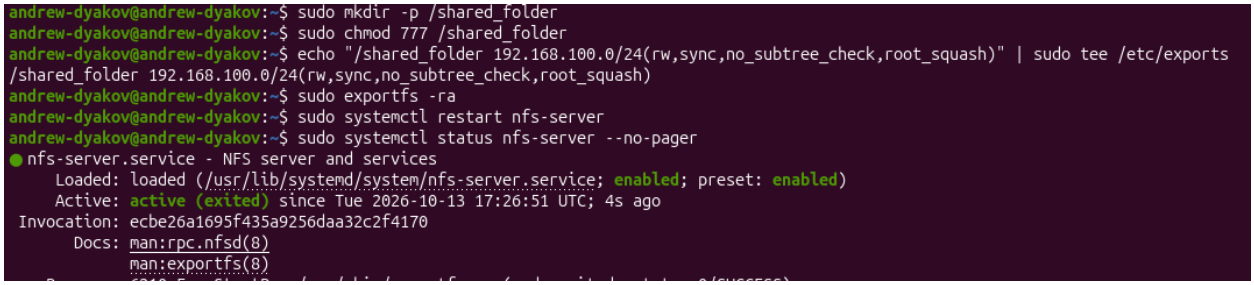


Рисунок 12 – Настройка NFS-сервера и экспорт каталога /shared_folder

На клиенте создан каталог /mnt/nfs_share и выполнено монтирование mount -t nfs 192.168.100.10:/shared_folder /mnt/nfs_share. Для проверки совместного доступа клиент создал файл test_nfs.txt и прочитал его (рисунок 13). Содержимое файла проверено и на стороне сервера (рисунок 14).

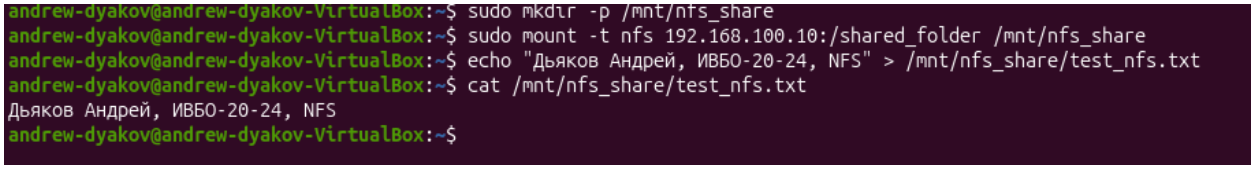


Рисунок 13 – Монтирование NFS-ресурса на клиенте и создание тестового файла

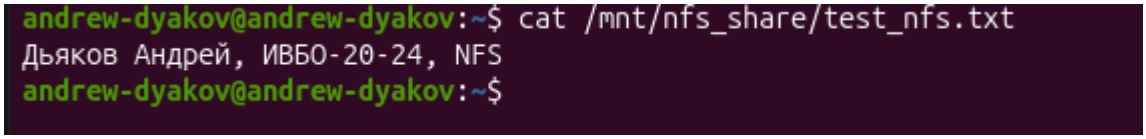


Рисунок 14 – Проверка файла test_nfs.txt на стороне сервера

1.4 Часть IV. Протокол SMB/CIFS

На сервере установлен пакет samba. Создан каталог /srv/samba/shared, его владельцем назначен пользователь andrew-dyakov, права установлены 770. Пользователь добавлен в базу паролей Samba (smbpasswd -a) и

активирован (smbpasswd -e), служба smbd включена в автозапуск и запущена командой systemctl enable --now smbd (рисунок 15).

```
andrew-dyakov@andrew-dyakov:~$ sudo mkdir -p /srv/samba/shared
andrew-dyakov@andrew-dyakov:~$ sudo chown andrew-dyakov:andrew-dyakov /srv/samba/shared
andrew-dyakov@andrew-dyakov:~$ sudo chmod 770 /srv/samba/shared
andrew-dyakov@andrew-dyakov:~$ sudo smbpasswd -a andrew-dyakov
New SMB password:
Retype new SMB password:
Added user andrew-dyakov.
andrew-dyakov@andrew-dyakov:~$ sudo smbpasswd -e andrew-dyakov
Enabled user andrew-dyakov.
andrew-dyakov@andrew-dyakov:~$ sudo systemctl enable --now smbd
Synchronizing state of smbd.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable smbd
```

Рисунок 15 – Подготовка общей папки и пользователя Samba, запуск службы smbd

В файл /etc/samba/smb.conf добавлена секция [SharedFolder]: путь /srv/samba/shared, запись разрешена (writable = yes), анонимный доступ запрещён (guest ok = no), подключаться может только andrew-dyakov (valid users), права новых файлов 0660, каталогов 0770 (рисунок 16). После редактирования служба smbd перезапущена (рисунок 17).

```
[SharedFolder]
   path = /srv/samba/shared
   writable = yes
   guest ok = no
   valid users = andrew-dyakov
   create mask = 0660
   directory mask = 0770
```

Рисунок 16 – Секция [SharedFolder] в файле /etc/samba/smb.conf

```
andrew-dyakov@andrew-dyakov:~$ sudo nano /etc/samba/smb.conf
andrew-dyakov@andrew-dyakov:~$ sudo systemctl restart smbd
```

Рисунок 17 – Редактирование smb.conf и перезапуск службы smbd

На клиенте создан каталог /mnt/cifs_share и выполнено монтирование командой mount -t cifs //192.168.100.10/SharedFolder /mnt/cifs_share -o username=andrew-dyakov,vers=3.0,uid=...,gid=...; пароль запрашивается интерактивно (рисунок 18). Эта запись эквивалентна mount.cifs ... -o user=... из методических указаний, при этом явно задана версия протокола SMB 3.0.

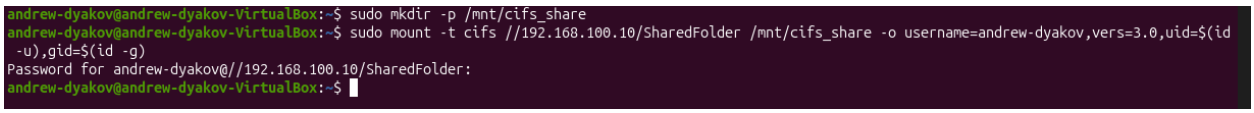


Рисунок 18 – Монтирование SMB-ресурса на клиенте

Для проверки клиент создал файл test_cifs.txt в смонтированном ресурсе и прочитал его. Затем выполнено подключение к серверу по SSH, и файл /srv/samba/shared/test_cifs.txt найден в локальной файловой системе сервера с тем же содержимым (рисунок 19).

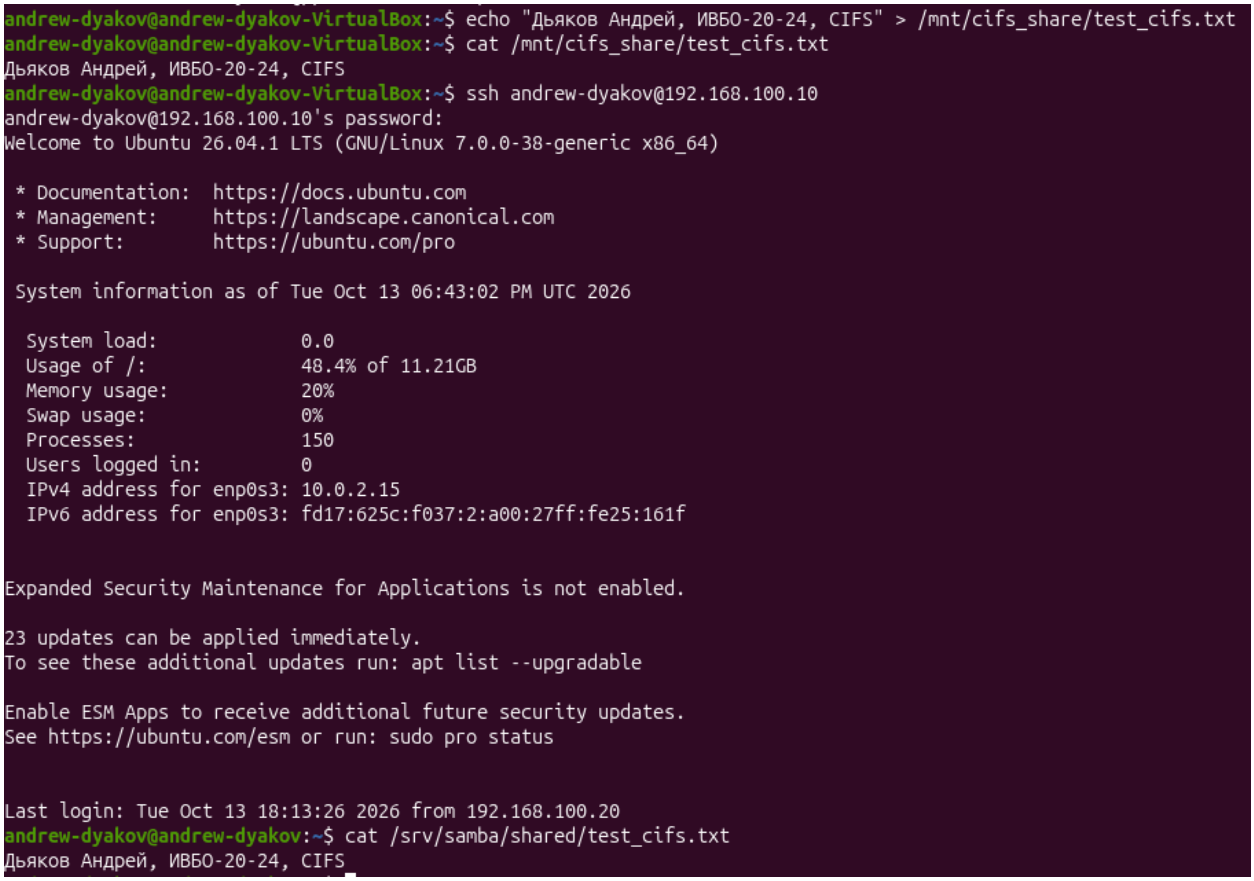


Рисунок 19 – Создание тестового файла на клиенте и проверка его на сервере

1.5 Часть V. Протоколы FTP и SFTP

На сервере установлены пакеты vsftpd и openssl (рисунок 20). В файле /etc/vsftpd.conf включён режим SSL/TLS: ssl_enable=YES, пути к сертификату /etc/ssl/certs/vsftpd.crt и ключу /etc/ssl/private/vsftpd.key, allow_anon_ssl=NO, force_local_data_ssl=YES, force_local_logins_ssl=YES, require_ssl_reuse=NO, ssl_tlsv1=YES (рисунок 21).

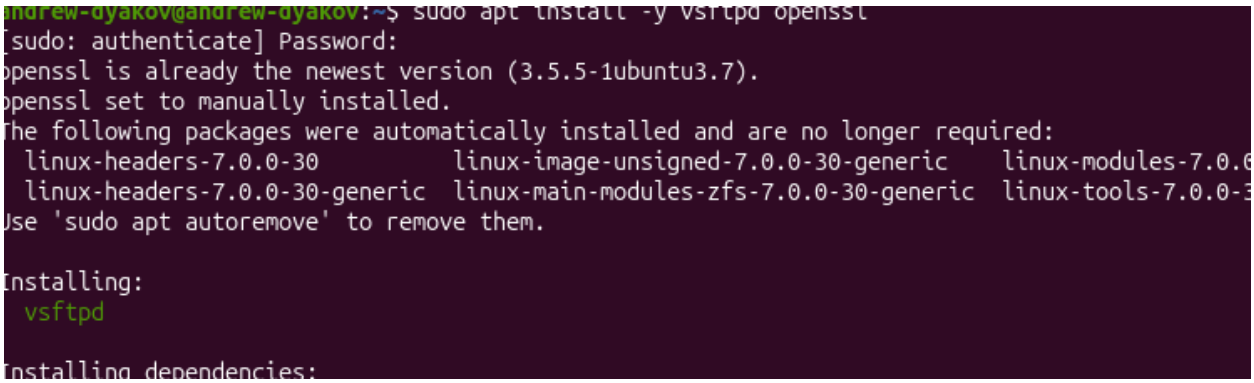


Рисунок 20 – Установка vsftpd и openssl на сервере

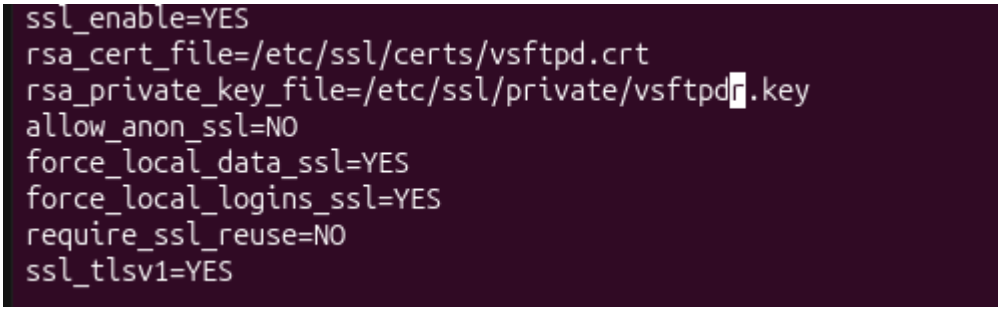


Рисунок 21 – Параметры SSL/TLS в файле /etc/vsftpd.conf

Самоподписанный сертификат и ключ сгенерированы командой openssl req -x509 -nodes -days 365 -newkey rsa:2048 (срок действия 365 дней, ключ RSA 2048 бит, CN=storage-server); права на закрытый ключ установлены 600, перед правкой сделана резервная копия vsftpd.conf (рисунок 22). Затем подготовлены каталог /run/vsftpd/empty и права домашнего каталога пользователя, после чего служба vsftpd перезапущена (рисунок 23).

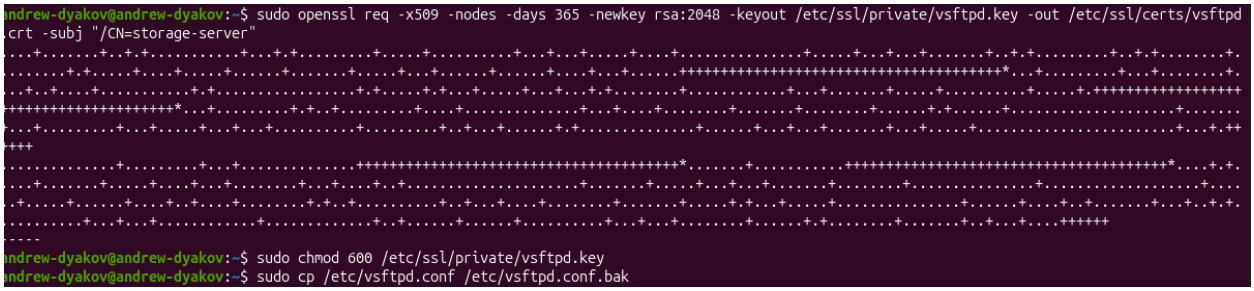


Рисунок 22 – Генерация сертификата и ключа, резервная копия конфигурации vsftpd

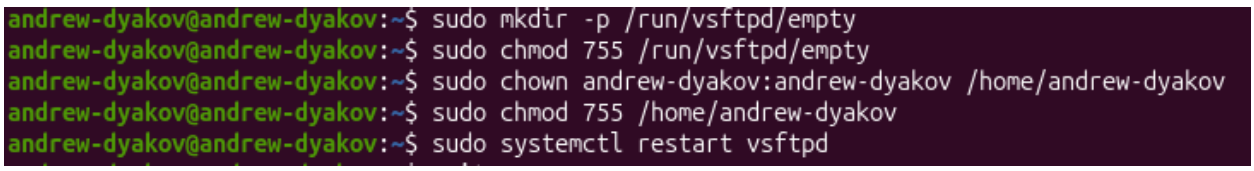


Рисунок 23 – Подготовка каталогов и перезапуск службы vsftpd

Для проверки FTP на клиенте использован lftp, так как стандартный клиент ftp не поддерживает TLS. Заданы принудительное шифрование управляющего канала и данных (ftp:ssl-force, ftp:ssl-protect-data); проверка сертификата отключена (ssl:verify-certificate no), поскольку сертификат самоподписанный. После авторизации пользователем andrew-dyakov командой put передан файл ftps_test.txt (48 байт), он отображён в каталоге сервера (ls -l) и прочитан командой cat (рисунок 24).

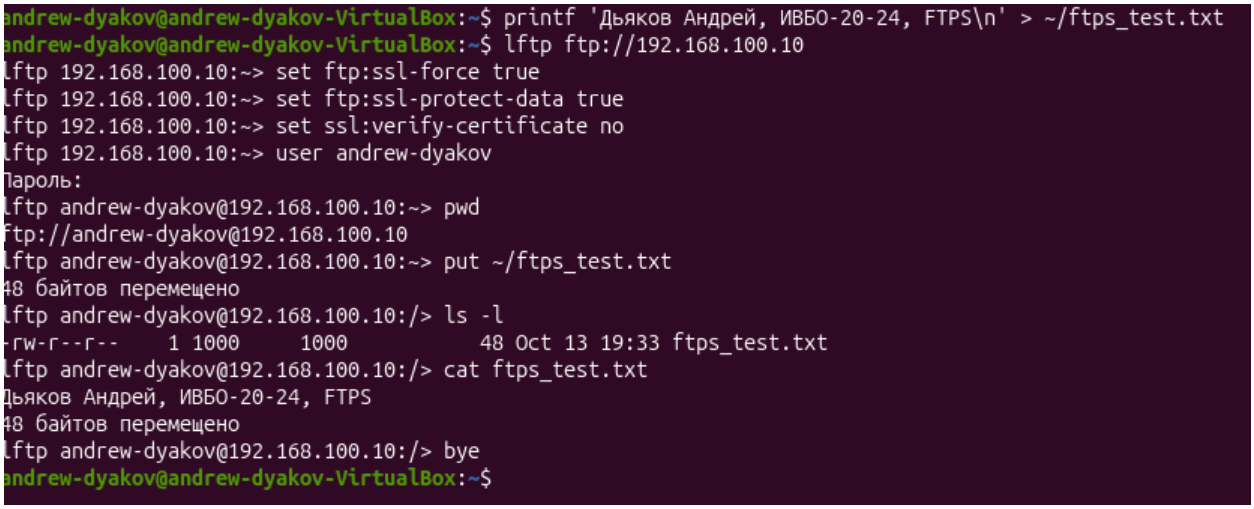


Рисунок 24 – Подключение к серверу по FTPS (lftp) и передача тестового файла

Для SFTP на сервере создан каталог ~/sftp_shared (рисунок 25). На клиенте установлен пакет sshfs (версия 3.7.3-1.1build5), создана точка монтирования ~/sftp_mount и выполнено монтирование sshfs andrew-dyakov@192.168.100.10:/home/andrew-dyakov/sftp_shared ~/sftp_mount. Команда findmnt -T подтверждает тип файловой системы fuse.sshfs. Файл test_sftp.txt, созданный в точке монтирования, прочитан на сервере командой ssh ... cat (рисунок 26).

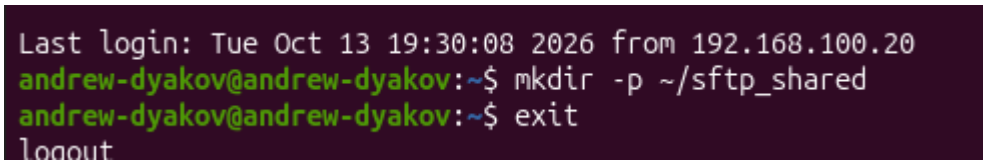


Рисунок 25 – Создание каталога sftp_shared на сервере

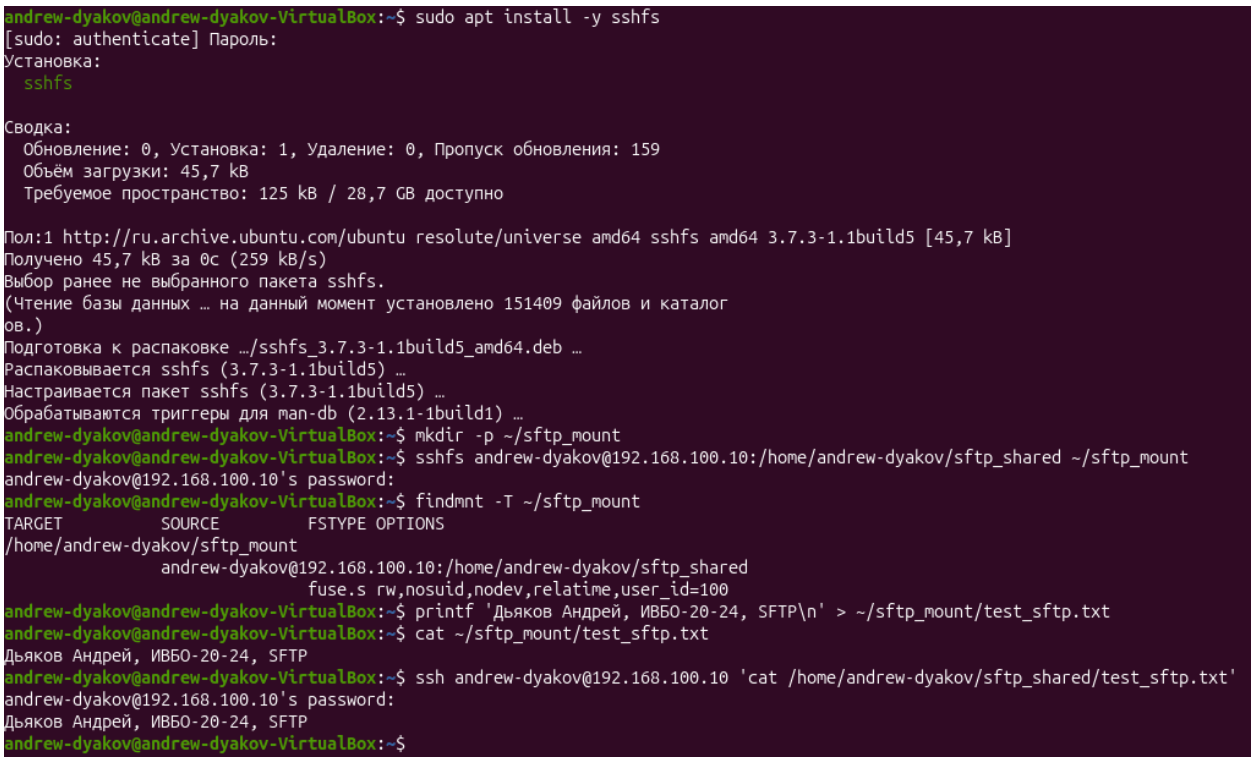


Рисунок 26 – Монтирование каталога сервера через SSHFS и проверка файла test_sftp.txt

2 СРАВНИТЕЛЬНЫЙ АНАЛИЗ ФАЙЛОВОГО, БЛОЧНОГО И ОБЪЕКТНОГО ТИПОВ ДОСТУПА

2.1 Блочный доступ

При блочном доступе клиенту предоставляется «сырое» устройство – последовательность блоков фиксированного размера (обычно 512 Б или 4 КиБ), адресуемых номером логического блока (LBA). Сервер хранения не знает о файлах и каталогах: файловую систему создаёт и обслуживает ОС клиента. В части II это видно напрямую: LUN 0 размером 5 ГБ появился на клиенте как локальный диск /dev/sde (рисунок 10), клиент самостоятельно выполнил на нём mkfs.ext4 и смонтировал в /mnt/iscsi (рисунок 11).

Реализация. Протокол iSCSI (RFC 7143) передаёт команды SCSI поверх TCP; target слушает порт 3260, адресуется по IQN формата iqn.ГГГГ-ММ.обратное-доменное-имя:идентификатор, а диск внутри target – по номеру LUN. На сервере блочное устройство предоставляет LIO (targetcli-fb), на клиенте – initiator open-iscsi (iscsiadm). Доступ разграничивается ACL по

IQN инициатора и CHAP-аутентификацией (рисунки 7, 9). Другие реализации блочного доступа: Fibre Channel, FCoE, NVMe over Fabrics.

Ключевое ограничение: обычные файловые системы (ext4, XFS, NTFS) рассчитаны на одного владельца. Если один LUN одновременно смонтировать на двух клиентах, метаданные будут повреждены, поэтому для совместного доступа нужны кластерные файловые системы (GFS2, OCFS2) или резервирования SCSI-3 (persistent reservations).

2.2 Файловый доступ

При файловом доступе файловой системой управляет сервер, а клиент работает с именованными файлами и каталогами через сетевой протокол (open, read, write, rename, lock). В работе файловый доступ реализован двумя протоколами: NFS (клиент смонтировал /shared_folder, рисунок 13) и SMB (ресурс SharedFolder, рисунок 18). Файл, созданный клиентом, сразу виден в локальной файловой системе сервера (рисунки 14, 19): данные принадлежат серверу, а не клиенту.

Преимущества: одновременная работа многих клиентов, права доступа и блокировки на уровне файлов, не нужно разбираться с разметкой диска. Недостатки: каждая операция над метаданными (open, stat, readdir) – сетевой запрос, а сервер файлового доступа (NAS) становится узким местом по производительности.

2.3 Объектный доступ

Объектное хранилище оперирует объектами, каждый из которых состоит из данных, уникального ключа и метаданных. Пространство имён плоское: «каталоги» в ключе – условность, объекты лежат в контейнерах (bucket). Доступ осуществляется по HTTP(S) через REST API (PUT, GET, DELETE, HEAD, LIST); де-факто стандартом стал API Amazon S3, который поддерживают также Ceph RGW и MinIO, а у OpenStack Swift есть

собственный API. Изменить часть объекта нельзя – он записывается заново целиком (или создаётся его новая версия).

Хранилище масштабируется горизонтально: объекты распределяются по узлам, надёжность обеспечивается репликацией или кодированием со стиранием (erasure coding). В Amazon S3 размер одного объекта достигает 5 ТБ. Платой за это становятся более высокая задержка (каждая операция – HTTP-запрос) и отсутствие блокировок и частичной перезаписи. Объектный доступ в рамках данной работы не настраивался; ближе всего к нему по принципу передачи файла целиком находится FTP/SFTP (рисунок 24), однако у FTP нет метаданных объектов, версионирования и масштабируемого плоского пространства имён.

2.4 Сравнение и области применения

Таблица 1 – Сравнение блочного, файлового и объектного доступа

Критерий	Блочный	Файловый	Объектный
Единица доступа	Блок (512 Б / 4 КиБ), адрес LBA	Файл в иерархии каталогов	Объект целиком: данные + ключ + метаданные
Кто управляет файловой системой	Клиент (ext4, XFS, NTFS)	Сервер хранения	Файловой системы нет, плоское пространство имён
Протоколы	iSCSI, Fibre Channel, FCoE, NVMe-oF	NFS, SMB	HTTP(S) REST: S3, Swift
Адресация	LUN, IQN / WWN	Путь к файлу, имя общего ресурса	Bucket + ключ объекта
Изменение данных	Запись любого блока	Запись в любое место файла	Только замена объекта целиком
Совместный доступ	Один владелец; несколько – только с кластерной ФС	Много клиентов, есть блокировки	Много клиентов, блокировок нет
Задержка	Минимальная	Средняя (метаданные по сети)	Наибольшая (HTTP-запрос на операцию)
Масштабирование	Ограничено ёмкостью массива и LUN	Ограничено сервером или кластером NAS	Практически неограниченное, горизонтальное

Критерий	Блочный	Файловый	Объектный
Типичные применения	Базы данных, виртуальные машины, загрузочные тома	Общие папки, домашние каталоги, данные для вычислительных кластеров	Резервные копии, архивы, медиа, журналы, статический контент
Пример в работе	iSCSI (часть II)	NFS и SMB (части III, IV)	Не входил в задание

Выбор типа доступа определяется нагрузкой:

– блочный доступ нужен там, где важны низкая задержка и предсказуемое число операций в секунду: базы данных, хранилища виртуальных машин, загрузочные тома. Цена – отсутствие совместного доступа и необходимость самостоятельно управлять файловой системой и резервным копированием;
– файловый доступ подходит для общих папок, домашних каталогов и данных, с которыми одновременно работает много пользователей или серверов; он удобен тем, что клиент видит привычные файлы и права;
– объектный доступ оправдан для больших объёмов неизменяемых данных (резервные копии, архивы, медиафайлы, журналы, озёра данных), но плохо подходит для баз данных и частых мелких изменений.

Облачные сервисы повторяют ту же классификацию: Amazon EBS – блочное хранилище (аналог SAN), Amazon EFS (NFS 4.1) и Azure Files (SMB) – файловое, Amazon S3 – объектное. На практике уровни комбинируют: NAS-шлюз экспортирует по NFS или SMB том, лежащий на LUN из SAN, а резервные копии выгружаются в объектное хранилище.

3 СРАВНИТЕЛЬНЫЙ АНАЛИЗ ПРОТОКОЛОВ NFS, CIFS, FTP

3.1 NFS

NFS (Network File System) – основной протокол сетевых файловых систем в UNIX и Linux. Версия 3 (RFC 1813) не хранит состояние сессии; блокировки вынесены в отдельные службы (NLM, NSM), а для монтирования нужны rpcbind (порт 111) и mountd, поэтому через межсетевой экран приходится пропускать несколько портов. Версии 4 (RFC 7530) и 4.1 (RFC 8881) работают с состоянием: блокировки и монтирование встроены в протокол, весь обмен идёт по TCP-порту 2049; версия 4.1 добавляет pNFS – параллельный доступ к данным на нескольких серверах.

По умолчанию (AUTH_SYS) сервер доверяет адресу клиента и идентификаторам UID/GID, которые тот сообщает, что не защищает от подмены. Надёжная аутентификация и шифрование обеспечиваются Kerberos (режимы krb5, krb5i, krb5p) или RPC-over-TLS (RFC 9289). Согласованность кэша NFS строится на модели close-to-open: при закрытии файла клиент записывает данные на сервер, при открытии – проверяет атрибуты. Поэтому одновременная запись нескольких клиентов в один файл без блокировок небезопасна.

В работе параметры экспорта rw,sync,no_subtree_check,root_squash означают: чтение и запись разрешены; сервер подтверждает запись после сохранения на диск; отключена проверка вхождения файла в экспортируемое поддерево; запросы root клиента выполняются от имени nobody. Доступ дополнительно ограничен подсетью 192.168.100.0/24 (рисунок 12).

3.2 CIFS/SMB

SMB (Server Message Block) – протокол Microsoft для доступа к файлам и принтерам. Название CIFS относится к диалекту SMB 1.0, который считается устаревшим и небезопасным и отключён по умолчанию в современных Windows; поэтому в работе клиент подключался по SMB 3.0 (vers=3.0)

драйвером cifs к серверу Samba (рисунок 18). Протокол использует TCP-порт 445 (в старых сетях – NetBIOS: порты 137, 138 UDP и 139 TCP), хранит состояние сессии и поддерживает блокировки диапазонов байт, oplocks и leases (кэширование на клиенте).

Аутентификация выполняется по логину и паролю (NTLMv2) либо через Kerberos в домене Active Directory. Samba хранит собственную базу паролей, поэтому пользователя ОС нужно отдельно добавить командой smbpasswd -a (рисунок 15). Шифрование появилось в SMB 3.0 (AES-128-CCM), в SMB 3.1.1 используется AES-128-GCM. Доступ к ресурсу в smb.conf ограничивается параметрами valid users (кто может подключаться), guest ok = no (запрет анонимного входа), writable, create mask и directory mask (права новых файлов 0660 и каталогов 0770; рисунок 16).

3.3 FTP, FTPS и SFTP

FTP (RFC 959) – протокол передачи файлов, а не сетевая файловая система: клиент получает список файлов, скачивает или загружает файл целиком, но не может открыть его в приложении и изменить часть данных. Используются два TCP-соединения: управляющее (порт 21) и канал данных. В активном режиме сервер сам подключается к клиенту со своего порта 20 на порт, сообщённый клиентом командой PORT, – такое соединение блокируют NAT и межсетевые экраны на стороне клиента. В пассивном режиме (PASV) сервер открывает случайный порт и сообщает его клиенту, клиент подключается сам; на сервере при этом нужно открыть диапазон портов (в vsftpd – pasv_min_port и pasv_max_port).

Обычный FTP передаёт логин и пароль открытым текстом. FTPS (RFC 4217) – тот же FTP, защищённый TLS: в явном режиме клиент на порту 21 командой AUTH TLS переходит на шифрование, в неявном используется порт 990. В работе включено ssl_enable=YES с принудительным TLS для входа и передачи данных (рисунок 21), применён самоподписанный сертификат (рисунок 22), поэтому клиент lftp запущен без проверки сертификата (рисунок

24); в рабочей среде необходим сертификат доверенного удостоверяющего центра.

SFTP не является вариантом FTP: это подсистема протокола SSH (порт 22), в которой и команды, и данные идут по одному зашифрованному соединению. В работе она использована через SSHFS, что позволило смонтировать удалённый каталог как локальный (рисунок 26).

3.4 Сравнение и различия в области применения

Таблица 2 – Сравнение протоколов NFS, CIFS/SMB и FTP/FTPS

Критерий	NFS	CIFS/SMB	FTP/FTPS
Тип протокола	Сетевая файловая система	Сетевая файловая система, общий доступ к принтерам	Передача файлов целиком
Стандарты и версии	v3 (RFC 1813), v4 (RFC 7530), v4.1 (RFC 8881)	SMB 1.0 (CIFS), 2.x, 3.0, 3.1.1	RFC 959; FTPS – RFC 4217
Порты по умолчанию	2049 (v4); для v3 также 111 и mountd	445/TCP (139 и 137–138 для NetBIOS)	21 (управление), 20 (данные, активный режим), 990 (неявный FTPS)
Состояние	v3 без состояния, v4 с состоянием	С состоянием	Состояние хранит управляющее соединение
Аутентификация	По адресу клиента и UID/GID; Kerberos	Логин и пароль (NTLMv2), Kerberos (AD)	Логин и пароль, анонимный вход
Шифрование	Нет по умолчанию; krb5p, RPC-over-TLS	Нет в SMB 1.0 и 2.x; AES-128-CCM/GCM в SMB 3	Нет в FTP; TLS в FTPS
Блокировки	Есть (в v4 встроены, в v3 – NLM)	Есть (диапазоны байт, oplocks, leases)	Нет
Частичное чтение и запись	Да	Да	Нет, только файл целиком (докачка командой REST)
Прохождение через NAT и межсетевой экран	v4 – один порт; v3 – несколько	Один порт 445	Сложно: второе соединение для данных

Критерий	NFS	CIFS/SMB	FTP/FTPS
Основные ОС	Linux, UNIX (в Windows – дополнительный компонент)	Windows; Linux и macOS (клиент и Samba)	Любые
Типичное применение	Серверы Linux, кластеры, хранилища виртуальных машин	Файловые серверы Windows и AD, общие папки	Обмен и публикация файлов, выгрузки
В работе	/shared_folder → /mnt/nfs_share (рис. 13)	SharedFolder → /mnt/cifs_share, vers=3.0 (рис. 18)	vsftpd + TLS, клиент lftp (рис. 24)

Различия в применении следуют из особенностей протоколов:

– NFS выбирают для сетей Linux и UNIX: он прост в настройке (одна строка в /etc/exports), обеспечивает прозрачный доступ к файлам и подходит для хранилищ виртуальных машин и кластеров, но без Kerberos его следует использовать только в доверенной сети;

– SMB необходим при работе с Windows и в доменах Active Directory: он даёт централизованную аутентификацию и шифрование (SMB 3), а Samba позволяет предоставить такие ресурсы с Linux-сервера;

– FTP/FTPS подходит для обмена файлами между организациями и публикации данных, когда монтирование ресурса не нужно; из-за двух соединений он хуже проходит через NAT, а без TLS небезопасен. Если нужен защищённый обмен, чаще выбирают SFTP: он использует один порт 22 и не требует отдельного сертификата.

ЗАКЛЮЧЕНИЕ

В ходе работы развёрнут виртуальный стенд из двух машин (клиент и сервер хранения данных) на Ubuntu 26.04.1 LTS. На сервере создан том ёмкостью 5 ГБ (LVM) и предоставлен клиенту по iSCSI с авторизацией CHAP: клиент обнаружил target, подключился, создал файловую систему ext4, смонтировал диск и записал тестовый файл. Файловый доступ проверен по NFS (экспорт, ограниченный подсетью стенда) и SMB 3.0 (Samba с

ограничением valid users): файлы, созданные на клиенте, подтверждены на сервере. Для передачи файлов настроены FTPS (vsftpd с TLS, самоподписанный сертификат) и SFTP через SSHFS.

Сравнительный анализ показал, что блочный доступ (iSCSI) обеспечивает минимальную задержку, но предполагает одного владельца тома; файловый (NFS, SMB) допускает совместную работу многих клиентов ценой накладных расходов на метаданные; объектный масштабируется лучше всех, но не поддерживает частичную перезапись и блокировки. Из протоколов файлового уровня NFS ориентирован на Linux, SMB – на Windows и домены AD, а FTP предназначен для передачи файлов и при использовании в сетях с NAT требует особой настройки или замены на SFTP.